

EXCEPTION HANDLING

Transcript-Based Notes • Simple Explanations • Box Diagrams • Interview Q&A;

CORE IDEA

A professional API should return the correct HTTP status and a predictable response body. Keep normal request processing simple, and handle exceptional situations in one central place.

CLIENT REQUEST

The frontend or Postman sends an HTTP request.



CONTROLLER → SERVICE

The controller passes the request to the service layer.



SUCCESS OR EXCEPTION

Return the normal result, or raise an exception when the operation cannot be completed.



HTTP RESPONSE

The client receives a meaningful status code and a consistent response.

Use this PDF for quick revision before Spring Boot interviews.

1. Anatomy of an API Response

An HTTP response has three key parts. They should agree with one another so the client can understand the result.

1. STATUS CODE

Explains the outcome: success, client-side problem, or server-side failure.



2. HEADERS

Metadata about the response, such as Content-Type.



3. BODY

The returned data or an error description, commonly represented as JSON.

Why this matters

PROFESSIONAL API

The status code and body tell the same story. For example, a missing student should not be returned as 200 OK with a message saying "not found."

Interview questions

Q1. What are the three parts of an HTTP response?

Status code, headers, and body.

Q2. Why is returning 200 for every outcome a bad practice?

The client cannot reliably distinguish success from failure using the status code.

Q3. What is the purpose of response headers?

They carry metadata about the response.

2. HTTP Status Codes — Quick Reference

Code	Meaning	When to use
200 OK	Request succeeded	Successful GET or another successful operation with a response body.
201 Created	Resource created	Student was successfully created.
400 Bad Request	Invalid client request	Invalid input, malformed data, or request validation failure.
404 Not Found	Resource missing	The requested student ID does not exist.
409 Conflict	Conflict with current state	A duplicate email where uniqueness is required.
500 Internal Server Error	Unexpected server failure	Unexpected bug or unhandled internal failure.

Status-code families

1xx — INFORMATIONAL

The request was received and processing continues.



2xx — SUCCESS

The request was successfully received, understood, and handled.



3xx — REDIRECTION

Further action is needed to complete the request.



4xx — CLIENT / REQUEST ERROR

The request has a problem or the requested resource cannot be found.



5xx — SERVER ERROR

The server failed to complete an apparently valid request.

Interview questions

Q1. Difference between 400 and 404?

400 means the request is invalid; 404 means the requested resource was not found.

Q2. When is 201 preferable to 200?

When a new resource has been successfully created.

Q3. Should an empty list from GET all students normally return 404?

No. A successful collection request can return 200 OK with an empty list.

3. RESTful Endpoint Design

Use the resource path to identify what is being managed and the HTTP method to express the operation. Avoid adding action words when the HTTP method already makes the action clear.

Operation	Method	Endpoint
Create a student	POST	/api/students
Get one student	GET	/api/students/{id}
Get all students	GET	/api/students
Update a student	PUT	/api/students/{id}
Delete a student	DELETE	/api/students/{id}
Partially update	PATCH	/api/students/{id} (when appropriate)

RESOURCE PATH

"/api/students" identifies the student collection.



HTTP METHOD

POST, GET, PUT, DELETE, or PATCH expresses the operation.



UNIQUE MAPPING

The combination of method and path distinguishes endpoints.

Interview questions

Q1. Why avoid /create and /get in REST endpoints?

The HTTP method already describes the operation, so action words are often redundant.

Q2. How can GET all students return no records successfully?

Return 200 OK with an empty array; the request itself succeeded.

Q3. What does PATCH generally represent?

A partial modification of a resource; its exact semantics depend on the API design.

4. Keep the Controller Thin

CONTROLLER

Receives HTTP input, delegates to the service, and returns the service result to the client. It should not be filled with repeated checks for every possible failure.

SERVICE

Contains business logic. It checks business conditions and can throw a meaningful exception when an operation cannot be completed.

REPOSITORY

Reads and writes data through the persistence layer, such as Spring Data JPA.

Normal flow vs exception flow

NORMAL / HAPPY FLOW

Client → Controller → Service → Repository → Service → Controller → Success response



EXCEPTION FLOW

Client → Controller → Service/Repository → Exception → Global handler → Error response

Why separate the flows?

- Avoid repeating the same if/else response logic in every controller method.
- Keep business rules out of controller methods.
- Make status codes and error bodies consistent across endpoints.

Interview questions

Q1. What is the controller's main responsibility?

Handle HTTP input/output and delegate application work to the service.

Q2. Where should business rules live?

Usually in the service layer.

Q3. Why centralize exception handling?

It reduces duplication and makes error responses consistent.

5. Missing Data: null Checks vs Custom Exceptions

REQUESTED STUDENT ID

The service tries to find the student.



STUDENT EXISTS?

If yes, map the entity to a response DTO and return it.



IF NOT FOUND

Raise a `ResourceNotFoundException` with a useful message.



GLOBAL HANDLER

Convert that exception into a 404 response.

SIMPLE EXPLANATION

Instead of returning null and forcing every controller to check it, the service can clearly report that the requested resource does not exist. The global handler decides how that failure is returned over HTTP.

Custom exception — concept

`ResourceNotFoundException`

A named exception used when a requested resource is absent. Example message: "Student with ID 100 was not found."

Interview questions

Q1. Why can returning null make code harder to maintain?

Every caller may need its own null check, and error handling can become inconsistent.

Q2. What is a custom exception?

An application-defined exception that communicates a specific failure meaning.

Q3. Where should a resource-not-found exception be raised?

Commonly in the service layer when the required entity cannot be found.

6. Global Exception Handling

EXCEPTION OCCURS

A service or repository operation cannot complete normally.



EXCEPTION PROPAGATES

The exception travels out of the normal controller/service flow.



@RestControllerAdvice

A central class provides exception-handling methods for REST controllers.



@ExceptionHandler

The matching handler creates an HTTP response for the exception type.



CLIENT GETS ERROR RESPONSE

The response has an appropriate status code and a clear body.

Key terms

Term	Simple meaning
@RestControllerAdvice	Centralized advice for REST controllers, including exception handling.
@ExceptionHandler	Connects a particular exception type to a handler method.
Global handler	The shared place that converts exceptions into HTTP responses.

Interview questions

Q1. What is @RestControllerAdvice?

A class-level mechanism for shared REST-controller behavior, commonly used for global exception handling.

Q2. What does @ExceptionHandler do?

Selects a method to handle a specified exception type.

Q3. Do you need to catch every exception inside each controller method?

No. A global handler can process exceptions that propagate from controller processing.

7. Consistent Error Response

A stable error structure makes client-side handling easier. Rather than returning unrelated response formats for different errors, use the same core fields where practical.

TIMESTAMP

When the error response was created.



STATUS + ERROR

Numeric HTTP status and short label, such as 404 / Not Found.



MESSAGE

A clear explanation suitable for the API client.



PATH

The endpoint path that produced the error.

Example shape — explained without code

404 NOT FOUND

Status: 404

Error: Not Found

Message: Student with ID 100 was not found

Path: /api/students/100

500 INTERNAL SERVER ERROR

Status: 500

Error: Internal Server Error

Message: Something went wrong. Please try again later.

Path: the request path

Safety rule

DO NOT LEAK INTERNAL DETAILS

For unexpected failures, do not expose stack traces, SQL details, secrets, or raw internal exception messages to the client. Log diagnostic details on the server and return a safe public message.

Interview questions

Q1. Why use a common error DTO?

Clients can parse errors consistently and display predictable messages.

Q2. Should the generic handler return raw exception messages?

Usually not; they may reveal internal details.

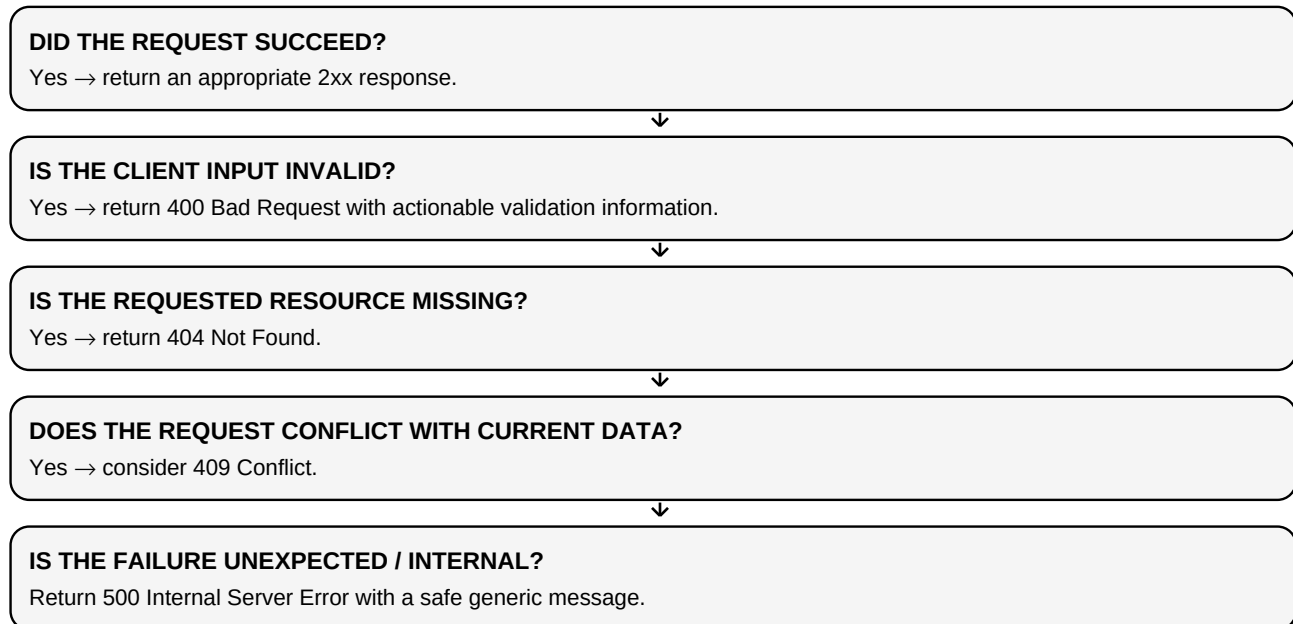
Q3. What useful fields can an error response include?

Timestamp, status, error label, message, and request path.

8. Common API Failure Scenarios

Scenario	Expected handling	Status
Student exists	Return the student response	200 OK
Student created	Return created resource/details	201 Created
Student ID does not exist	Resource-not-found handling	404 Not Found
Invalid request data	Validation / bad-request handling	400 Bad Request
Duplicate email	Business conflict handling	409 Conflict
Unexpected application bug	Safe generic error response	500 Internal Server Error

Decision flow



Interview questions

Q1. Why not use 500 for every failure?

Known client errors and missing resources have more accurate status codes.

Q2. When can 409 Conflict be appropriate?

When the request conflicts with the current state, such as a uniqueness constraint.

Q3. What should the client see for an unexpected server bug?

A generic, safe message; detailed diagnostics should remain in server logs.

9. Request Validation

@Valid

When used on a request body, it asks Spring to validate the incoming DTO using its validation constraints.

VALIDATION FAILURE

Spring can raise `MethodArgumentNotValidException` when request-body fields fail validation.

GLOBAL VALIDATION HANDLER

Collect field-level errors and return 400 Bad Request with a readable validation response.

Good validation response

VALIDATION FAILED

Status: 400

Error: Validation Failed

Field errors:

- name — Name is required
- age — Student must be at least 18 years old
- email — Student email must be valid

Good practices

- Tell the client which field needs correction.
- Use clear messages rather than returning the full framework exception text.
- Keep the error response predictable.
- Use the constraints that match the actual business rule.

Interview questions

Q1. What is `MethodArgumentNotValidException`?

An exception commonly raised when `@Valid` request-body validation fails.

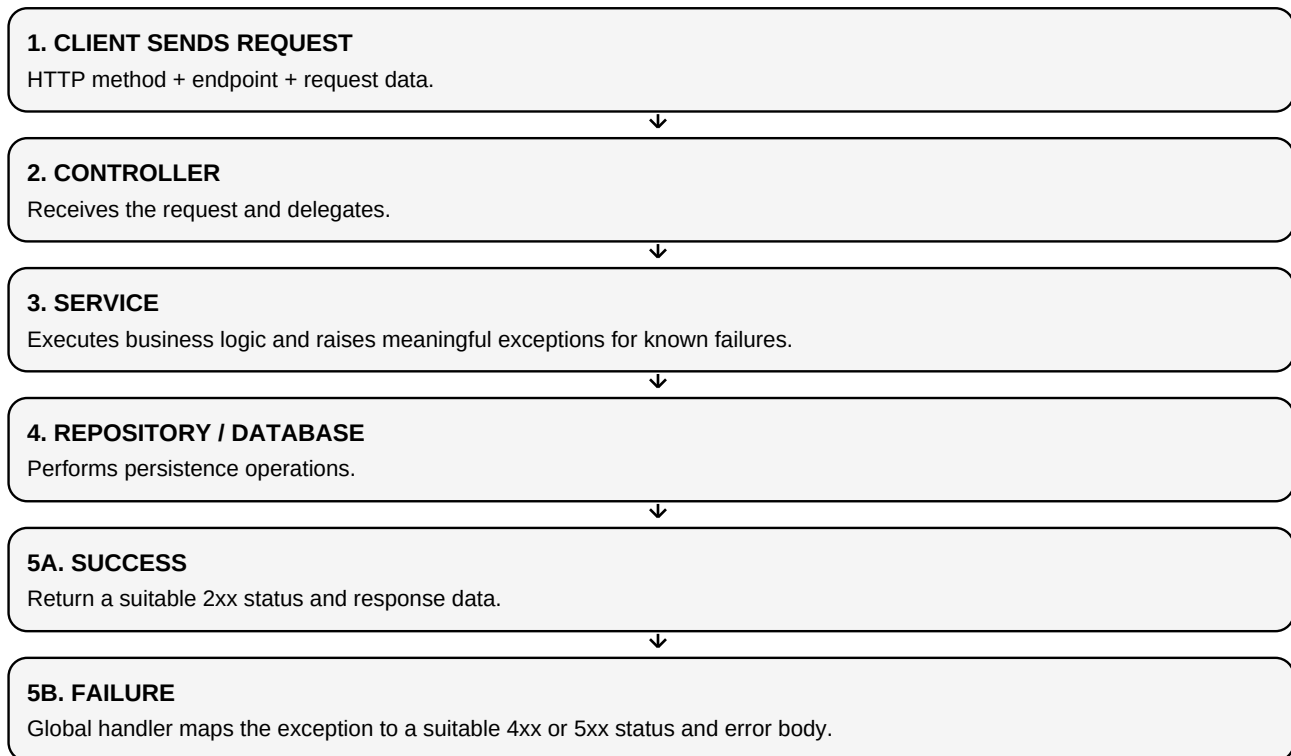
Q2. Which status code is suitable for invalid request fields?

400 Bad Request.

Q3. Why return field-level validation messages?

The client can identify and correct each invalid field.

10. Final Revision Map



Do not forget

- 404 = Not Found; 400 = Bad Request.
- Use 201 Created when a resource is created successfully.
- An empty collection response is usually 200 with an empty list.
- Keep controllers focused on HTTP input/output.
- Use `@RestControllerAdvice` and `@ExceptionHandler` for centralized error handling.
- Return consistent error bodies; keep sensitive diagnostic details in server logs.

Final interview drill

Q1. Explain exception handling in Spring Boot in 30 seconds.

It maps application failures to meaningful HTTP responses. Services raise meaningful exceptions, and a global handler converts them into consistent response bodies with suitable status codes.

Q2. Describe the difference between the happy path and exception path.

The happy path returns normal results; the exception path routes failures to centralized handling.

Q3. What makes an API error response professional?

Accurate status code, consistent body structure, useful client-safe message, and no leaked internal details.

ONE-LINE SUMMARY

Keep the happy path clean, use meaningful exceptions for known failures, and centralize exception-to-response mapping with a global handler.